Speaking Script – Evaluating LLM Pruning Techniques for Efficient Edge Deployment

Oct 31, 2026 · @Nitk Manoj

Timing plan

Slides 1–10 take 8 minutes, and slide 11 is the 2-minute summary. The script below is about 900 words for the talk and 215 for the summary, which fits a calm pace of about 115 words a minute. Text in *[brackets]* is an action, not something to say.

Slide	Topic	Start – end	Length
1	Title and greeting	0:00 – 0:30	30 s
2	Introduction	0:30 – 1:45	75 s
3	Related work	1:45 – 2:45	60 s
4	Three pruning techniques	2:45 – 4:15	90 s
5	Illustration of the methods	4:15 – 5:00	45 s
6	Experimental setup	5:00 – 5:45	45 s
7	Results: OPT	5:45 – 6:35	50 s
8	Results: BLOOM	6:35 – 7:05	30 s
9	Results: GPT-Neo	7:05 – 7:35	30 s
10	Downstream benchmarks	7:35 – 8:00	25 s
11	Summary (conclusion)	8:00 – 9:45	105 s
12	Thank you (references on screen)	9:45 – 10:00	15 s

If you are running late at slide 6, shorten slides 5 and 8 to one sentence each. Do not cut the summary.

Script: slides 1–3

Slide 1 – Title (0:00 – 0:30)

Good morning, everyone. Thank you to the session chair. My name is Rajarajeshwari Chanda, from the Department of Computer Science and Engineering, NIT Hamirpur.

Today I will present our paper, “Evaluating LLM Pruning Techniques for Efficient Edge Deployment.” This work was done with Mahroosh Banday from NIT Srinagar, Omais Shafi from ARM Research, Cambridge, and Mohammad Khalid Pandit from NIT Hamirpur.

[Change “Good morning” to “Good afternoon” if needed.]

Slide 2 – Introduction (0:30 – 1:45)

Let me start with the problem. Large language models, or LLMs, are used in almost every language application today: chatbots, translation, search and more. But these models are very big. They need a lot of memory and a lot of computing power. So it is hard to run them on edge devices, like phones, cars or small embedded boards.

Why do we want them on the edge? Because when the model runs on the device, the user’s data stays private, and it works even without the internet.

One way to make a model smaller is pruning. In pruning, we remove the weights that are not very important.

In this paper, we do four things. First, we compare three pruning methods: Magnitude pruning, SparseGPT and Wanda. Second, we test them on three model families, OPT, BLOOM and GPT-Neo, at different sizes. Third, we study how performance changes as we prune more and more. And finally, we give simple guidelines for choosing a pruning method.

Slide 3 – Related work (1:45 – 2:45)

There are three main ways to compress an LLM.

Quantization stores each weight with fewer bits, for example 8 or 4 bits instead of 16. Knowledge distillation trains a small “student” model to copy a big “teacher” model. And pruning, which is our focus, removes the least important weights.

Pruning also has three types. Structured pruning removes whole neurons or layers, so it is easy for hardware to speed up. Unstructured pruning removes single weights from anywhere, so it keeps accuracy better, but the result is a sparse matrix. Semi-structured pruning is in between. It follows a fixed pattern, like 2:4 sparsity, where two out of every four weights are zero. *[Say “two-four sparsity.”]*

In this work, we focus on unstructured pruning.

Script: slides 4–5

Slide 4 – Pruning techniques (2:45 – 4:15)

Now, the three pruning methods we compare. Each method gives every weight a score. Then it removes the weights with the lowest scores.

[Point to the left box.] The first is Magnitude pruning. The score is simply the absolute value of the weight. The idea is that small weights matter least. It is the simplest and fastest method, and it needs no data.

[Point to the middle box.] The second is SparseGPT. It uses a small set of input data to estimate how much the output will change if we remove a weight. This uses second-order information, called the Hessian. Its key strength is that after removing weights, it updates the remaining weights to make up for the loss. It works in one shot, with no retraining, and it scales to models with more than 100 billion parameters.

[Point to the right box.] The third is Wanda. Wanda multiplies the size of each weight by the size of its input activation. So a small weight can still be kept if its input is large. Wanda compares weights within each output row. It does not retrain or update any weights, so it is very fast.

Slide 5 – Illustration of the methods (4:15 – 5:00)

This slide shows each method on a small example.

[Point to Fig. 1, top left.] In Magnitude pruning, we take the absolute value of each weight and remove the smallest ones.

[Point to Fig. 3, bottom left.] In Wanda, each weight is multiplied by its input activation norm. Look at the first row. The weight minus 3 is small, so Magnitude pruning removes it. But Wanda keeps it, because its input value, 8, is large.

[Point to Fig. 2, right.] SparseGPT works column by column. It removes some weights, and then updates the remaining weights, shown in dark blue, to correct the error.

Script: slides 6–10

Slide 6 – Experimental setup (5:00 – 5:45)

Here is our setup. We used three model families: OPT, from 125 million to 2.7 billion parameters; BLOOM, from 560 million to 1.7 billion; and GPT-Neo, from 125 million to 2.7 billion.

For calibration, we used 128 text sequences from the C4 dataset. We measured perplexity on the WikiText-2 validation set. Perplexity tells us how well the model predicts real text. Lower is better. We pruned each model from 0 to 90 percent.

We also tested accuracy on three reasoning tests: ARC-Easy, ARC-Challenge and HellaSwag.

Slide 7 – Results: OPT (5:45 – 6:35)

Now the results. This chart shows perplexity for OPT-2.7B. The y-axis is a log scale, and lower is better.

At low sparsity, 10 to 30 percent, all three methods are close to the original model, and Wanda is slightly the best.

[Point to the dark blue line.] From 40 percent, SparseGPT is the best. It stays almost the same as the original model up to about 60 percent.

[Point to the grey line.] Magnitude pruning breaks down after 30 percent. At 60 percent, its perplexity is in the thousands.

We also saw that larger OPT models handle pruning better than smaller ones.

Slide 8 – Results: BLOOM (6:35 – 7:05)

BLOOM-1.1B shows a similar pattern. Wanda is best at 10 to 30 percent. SparseGPT leads from 50 percent and above. Magnitude pruning is usable only up to about 20 percent. At 80 to 90 percent, some settings cross a perplexity of 100,000, so the model is no longer useful.

Slide 9 – Results: GPT-Neo (7:05 – 7:35)

For GPT-Neo-1.3B, SparseGPT is the best at every level. It stays usable where the other methods fail. Magnitude pruning drops the most: its perplexity goes above one million. So for GPT-Neo, SparseGPT is clearly the top choice.

Slide 10 – Downstream benchmarks (7:35 – 8:00)

Finally, we checked real task accuracy, using SparseGPT on OPT-125M and OPT-350M. Accuracy stays almost flat up to about 40 percent, and then slowly drops. The bigger model, OPT-350M, starts higher on all three tests and holds up better overall.

Script: 2-minute summary (slides 11–12)

Slide 11 – Conclusion (8:00 – 9:45)

Let me now summarise our work. We compared three pruning methods on three LLM families, from 0 to 90 percent sparsity.

[Point to the SparseGPT box.] SparseGPT was the most consistent method. It stayed close to the original model up to about 60 percent sparsity. Even at 80 to 90 percent, it was still the most usable. This is because it updates the remaining weights after pruning.

[Point to the Wanda box.] Wanda was the best at light pruning, 10 to 30 percent. It gives a good balance between quality and pruning time, because it does not update any weights.

[Point to the Magnitude box.] Magnitude pruning is the simplest, but it breaks down beyond about 20 percent sparsity.

[Point to Takeaways.] We have two main takeaways. First, 50 percent sparsity works for most tasks with only a small loss. That means we can remove half of the weights. Second, larger models have more extra weights, so they can be pruned more.

So our simple guideline is: for light pruning, use Wanda. For heavy pruning, use SparseGPT. Use Magnitude pruning only when you need something very simple.

[Point to Limitations.] Our work has one limitation. We studied only unstructured sparsity. In future work, we will study semi-structured 2:4 sparsity, which real hardware can speed up more easily.

Slide 12 – Thank you (9:45 – 10:00)

[Move to the references slide.] These are the references for our work. Thank you for listening. I am happy to take your questions.

If you have only 30 seconds left: “To summarise: SparseGPT is the safest choice and keeps quality up to about 60 percent sparsity. Wanda is best for light pruning. Magnitude pruning fails early. Half the weights can be removed with only a small loss. Thank you.”

Delivery tips and likely questions

Tips for the online talk

- Test your microphone and screen share 10 minutes before your slot.
- Keep a timer in view. At slide 6 you should be at about 5:00.
- Look at the camera, not at your own face on screen.
- Pause for one breath after each slide title. It helps listeners follow.

- Say “SparseGPT” as “Sparse G-P-T”, and “Wanda” like the name.

Likely questions with simple answers

Why does SparseGPT do better at high sparsity? Because after it removes weights, it updates the remaining weights to fix the error. Magnitude and Wanda only remove weights; they do not correct anything.

Does pruning make the model faster on an edge device? In this paper we measured quality, not speed. Unstructured sparsity mainly saves memory. To get real speed-ups, the hardware needs a regular pattern like 2:4 sparsity, which is our future work.

Why only models up to 2.7 billion parameters? Our focus is edge deployment, and models of this size are closer to what can run on edge devices.

Why does ARC-Challenge stay flat on slide 10? These small models already score very low on this hard test, so there is little left to lose when we prune.

Can pruning be combined with quantization? Yes. They work in different ways and can be used together. We did not test that combination in this paper.